02 — Parametric Model Experimentation

Rx Drug MMIX Analysis

This notebook builds and benchmarks three parametric models to predict pharmaceutical sales from marketing mix variables. The modeling approach is **progressive and diagnostic**: each specification is motivated by findings from the previous one.

From EDA (Notebook 01), the specification is grounded in:

- Log-log functional form (justified by skewness reduction and linearization)
- Agent and year controls (justified by 50× baseline variation across agents, non-linear time trends)
- Class-level interaction terms (justified by faceted response slope differences)
- Awareness that 47% of observations have zero DTCA (concentrated in SSRIs)
- Diminishing returns in detailing, with saturation onset at $80–93M

Three parametric models:

1. **Pooled OLS** — baseline sanity check
2. **Fixed Effects OLS** with class interactions and DTCA regime indicator — primary candidate
3. **Mixed Effects** with random intercepts by agent — panel-aware alternative

Data split: Time-based (Train ≤ 2020, Validation 2021–2022, Test 2023–2024)

Units: Sales (B), $Detailing$(M), DTCA ($M)

---

0. Setup

In [1]:
```
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
import statsmodels.formula.api as smf
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

sns.set_theme(style="whitegrid")
pd.set_option("display.max_columns", 100)
pd.set_option("display.float_format", lambda x: f"{x:,.4f}")

print("Libraries loaded.")
```

```
Libraries loaded.
```

---

1. Load and Validate Processed Data

In [3]:
```
df = pd.read_csv("data/processed_data.csv")
print(f"Shape: {df.shape}")
print(f"Columns: {df.columns.tolist()}")
print(f"Years: {df['year'].min()} to {df['year'].max()}")
print(f"Classes: {sorted(df['class'].unique())}")
print(f"Agents: {df['agent'].nunique()}")
df.head()
```

```
Shape: (207, 11)
Columns: ['class', 'agent', 'year', 'detailing', 'direct_to_consumer_ad', 'actual_sales', 'detailing_buc
ket', 'dtca_zero', 'log_sales', 'log_detailing', 'log_dtca']
Years: 2013 to 2024
Classes: ['PPI', 'SSRI', 'Statin']
Agents: 22
```

Out[3]:

	class	agent	year	detailing	direct_to_consumer_ad	actual_sales	detailing_bucket	dtca_zero	log_sales	log_
0	PPI	Prevacid	2013	43.8805	0.0000	2.9083	(28.631, 44.969]	1	1.3631	
1	PPI	Prilosec	2013	41.6480	0.0000	14.7235	(28.631, 44.969]	1	2.7552	
2	SSRI	Luvox	2013	22.3667	0.0000	0.9120	(6.907, 28.631]	1	0.6482	
3	SSRI	Paxil	2013	57.4608	0.0000	11.2677	(44.969, 67.258]	1	2.5070	
4	SSRI	Prozac	2013	84.5333	0.0000	20.2600	(80.046, 92.864]	1	3.0568	

---

2. Feature Engineering for Parametric Modeling

The EDA prepared `log_sales`, `log_detailing`, `log_dtca`, and `dtca_zero`. Several additional features are required for robust parametric modeling:

Feature	Rationale
`year_trend`	Continuous year variable (year – 2013) replacing year dummies. Year dummies cannot predict on unseen future years, which breaks validation and test evaluation. A linear trend captures the secular decline identified in the EDA.
`dtca_zero`	Binary indicator for the zero-DTCA regime (47% of observations). The EDA established that this reflects a deliberate physician-only strategy, not under-investment. The log transform alone does not distinguish "chose not to advertise" from "advertised a tiny amount."
Class × channel interactions	Handled via the formula API. The EDA showed Statins have the steepest detailing response and SSRIs the flattest; a pooled coefficient would mask these differences.
`agent` (as grouping variable)	Used for fixed effects (Model 2) or random intercepts (Model 3) to absorb the 50× baseline variation across agents identified in the EDA.

In [4]:

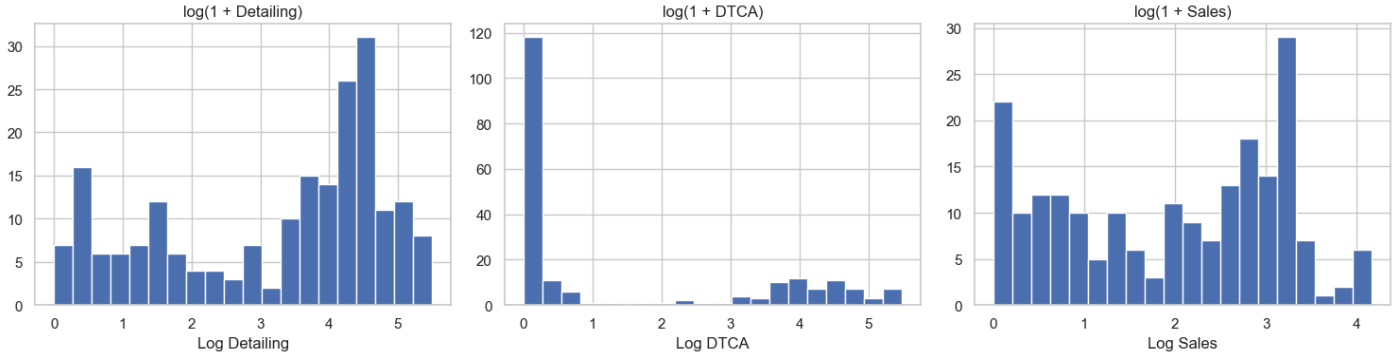
```
# Verify log-transformed variables exist; recreate if needed
for col, src in [("log_sales", "actual_sales"), ("log_detailing", "detailing"), ("log_dtca", "direct_to
    if col not in df.columns:
        df[col] = np.log1p(df[src])

# Verify dtca_zero indicator
if "dtca_zero" not in df.columns:
    df["dtca_zero"] = (df["direct_to_consumer_ad"] == 0).astype(int)

# Create continuous year trend (centered at 2013 to improve interpretability)
df["year_trend"] = df["year"] - 2013

# Strip leading/trailing whitespace from string columns (prevents Patsy KeyErrors)
df["agent"] = df["agent"].str.strip()
df["class"] = df["class"].str.strip()

print("Feature engineering complete.")
print(f"\nZero-DTCA share: {df['dtca_zero'].mean():.1%}")
print(f"Year trend range: {df['year_trend'].min()} to {df['year_trend'].max()}")
print(f"\nNew columns: year_trend")
print(f"Verified columns: log_sales, log_detailing, log_dtca, dtca_zero")
```

```
Feature engineering complete.

Zero-DTCA share: 47.3%
Year trend range: 0 to 11

New columns: year_trend
Verified columns: log_sales, log_detailing, log_dtca, dtca_zero
```

In [5]:

```
# Quick distribution check on engineered features
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

axes[0].hist(df["log_detailing"], bins=20, edgecolor="white")
axes[0].set_title("log(1 + Detailing)")
axes[0].set_xlabel("Log Detailing")

axes[1].hist(df["log_dtca"], bins=20, edgecolor="white")
axes[1].set_title("log(1 + DTCA)")
axes[1].set_xlabel("Log DTCA")

axes[2].hist(df["log_sales"], bins=20, edgecolor="white")
```

```
axes[2].set_title("log(1 + Sales)")
axes[2].set_xlabel("Log Sales")

plt.tight_layout()
plt.show()

print("\n[Validation] Skewness after log transformation:")
for col in ["log_sales", "log_detailing", "log_dtca"]:
    print(f"  {col}: {df[col].skew():.3f}")
```




```
[Validation] Skewness after log transformation:
  log_sales: -0.156
  log_detailing: -0.608
  log_dtca: 0.826
```

---

3. Time-Based Data Split

The data is time-indexed panel data. A random split would leak future information into training. We use a time-based split that reflects real forecasting conditions:

Split	Years	Purpose
Train	2013–2020	Build the model on historical data
Validation	2021–2022	Select the best specification
Test	2023–2024	Final holdout, evaluated only once

Cold-start handling: Pexeva enters the dataset in 2021 (validation period) with no training history. For Model 2 (agent fixed effects), Pexeva observations are excluded from out-of-sample evaluation since the model has no learned intercept for this agent. For Model 3 (mixed effects), Pexeva receives the population-level intercept (random effect = 0), which is a principled default.

In [6]:
```
train_df = df[df["year"] <= 2020].copy()
valid_df = df[(df["year"] >= 2021) & (df["year"] <= 2022)].copy()
test_df  = df[df["year"] >= 2023].copy()

print(f"Train:      {train_df.shape[0]:>4} rows  |  years {sorted(train_df['year'].unique())}")
print(f"Validation: {valid_df.shape[0]:>4} rows  |  years {sorted(valid_df['year'].unique())}")
print(f"Test:       {test_df.shape[0]:>4} rows  |  years {sorted(test_df['year'].unique())}")

# Identify cold-start agents
train_agents = set(train_df["agent"].unique())
valid_agents = set(valid_df["agent"].unique())
test_agents  = set(test_df["agent"].unique())

cold_valid = valid_agents - train_agents
cold_test  = test_agents - train_agents

print(f"\nAgents in validation but not train (cold-start): {cold_valid or 'None'}")
print(f"Agents in test but not train (cold-start):       {cold_test or 'None'}")

# Prepare filtered sets for Model 2 (FE), excluding cold-start agents
valid_df_fe = valid_df[valid_df["agent"].isin(train_agents)].copy()
test_df_fe  = test_df[test_df["agent"].isin(train_agents)].copy()

print(f"\nFiltered validation (excl. cold-start): {valid_df_fe.shape[0]} rows")
print(f"Filtered test (excl. cold-start):       {test_df_fe.shape[0]} rows")
```

```
Train:       119 rows  |  years [np.int64(2013), np.int64(2014), np.int64(2015), np.int64(2016), np.int6
4(2017), np.int64(2018), np.int64(2019), np.int64(2020)]
Validation:   44 rows  |  years [np.int64(2021), np.int64(2022)]
Test:         44 rows  |  years [np.int64(2023), np.int64(2024)]

Agents in validation but not train (cold-start): {'Pexeva'}
Agents in test but not train (cold-start):       {'Pexeva'}

Filtered validation (excl. cold-start): 42 rows
Filtered test (excl. cold-start):       42 rows
```

---

4. Evaluation Utilities

All models predict on the log scale. Evaluation is performed on the original dollar scale to ensure business-relevant metrics. MAPE is computed only for observations with actual sales above $0.01B to avoid division by near-zero values.

In [7]:
```
def evaluate(y_true, y_pred_log, model_name, dataset_name):
    """Convert log predictions back to original scale and compute metrics."""
    y_pred = np.expm1(np.array(y_pred_log))
    y_pred = np.maximum(y_pred, 0)
    y_true = np.array(y_true)

    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    mae  = mean_absolute_error(y_true, y_pred)
    r2   = r2_score(y_true, y_pred)

    mask = y_true > 0.01
    mape = np.mean(np.abs((y_true[mask] - y_pred[mask]) / y_true[mask])) * 100

    return pd.DataFrame({
        "Model": [model_name],
        "Dataset": [dataset_name],
        "RMSE ($B)": [round(rmse, 4)],
        "MAE ($B)":  [round(mae, 4)],
        "R²":        [round(r2, 4)],
        "MAPE (%)": [round(mape, 2)]
    })


def plot_actual_vs_predicted(y_true, y_pred_log, title):
    """Scatter of actual vs predicted on original scale."""
    y_pred = np.expm1(np.array(y_pred_log))
    plt.figure(figsize=(7, 6))
    plt.scatter(y_true, y_pred, alpha=0.6, edgecolors="white", linewidth=0.5)
    lims = [0, max(np.max(y_true), np.max(y_pred)) * 1.05]
    plt.plot(lims, lims, "--", color="gray", linewidth=1, label="Perfect prediction")
    plt.xlabel("Actual Sales ($B)")
    plt.ylabel("Predicted Sales ($B)")
    plt.title(title)
    plt.legend()
    plt.tight_layout()
    plt.show()


def plot_residual_diagnostics(fitted, residuals, title_prefix):
    """Residuals vs fitted and Q-Q plot side by side."""
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    axes[0].scatter(fitted, residuals, alpha=0.5, edgecolors="white", linewidth=0.5)
    axes[0].axhline(0, linestyle="--", color="gray")
    axes[0].set_xlabel("Fitted Values (log scale)")
    axes[0].set_ylabel("Residuals")
    axes[0].set_title(f"{title_prefix}: Residuals vs Fitted")

    sm.qqplot(residuals, line="45", ax=axes[1])
    axes[1].set_title(f"{title_prefix}: Q-Q Plot")

    plt.tight_layout()
    plt.show()


print("Evaluation utilities ready.")
```

```
Evaluation utilities ready.
```

---

5. Model 1 — Pooled OLS (Baseline)

The simplest specification: regress log(sales) on the two marketing variables plus class dummies, a linear year trend, and the DTCA regime indicator. No agent-level controls, no interactions.

Purpose: Establish a baseline to measure how much the more complex specifications improve fit. Residual diagnostics from this model will motivate the additions in Model 2.

Specification:

$$\log(1 + \text{Sales}) \sim \log(1 + \text{Detailing}) + \log(1 + \text{DTCA}) + \text{dtca_zero} + C(\text{class}) + \text{year_trend}$$

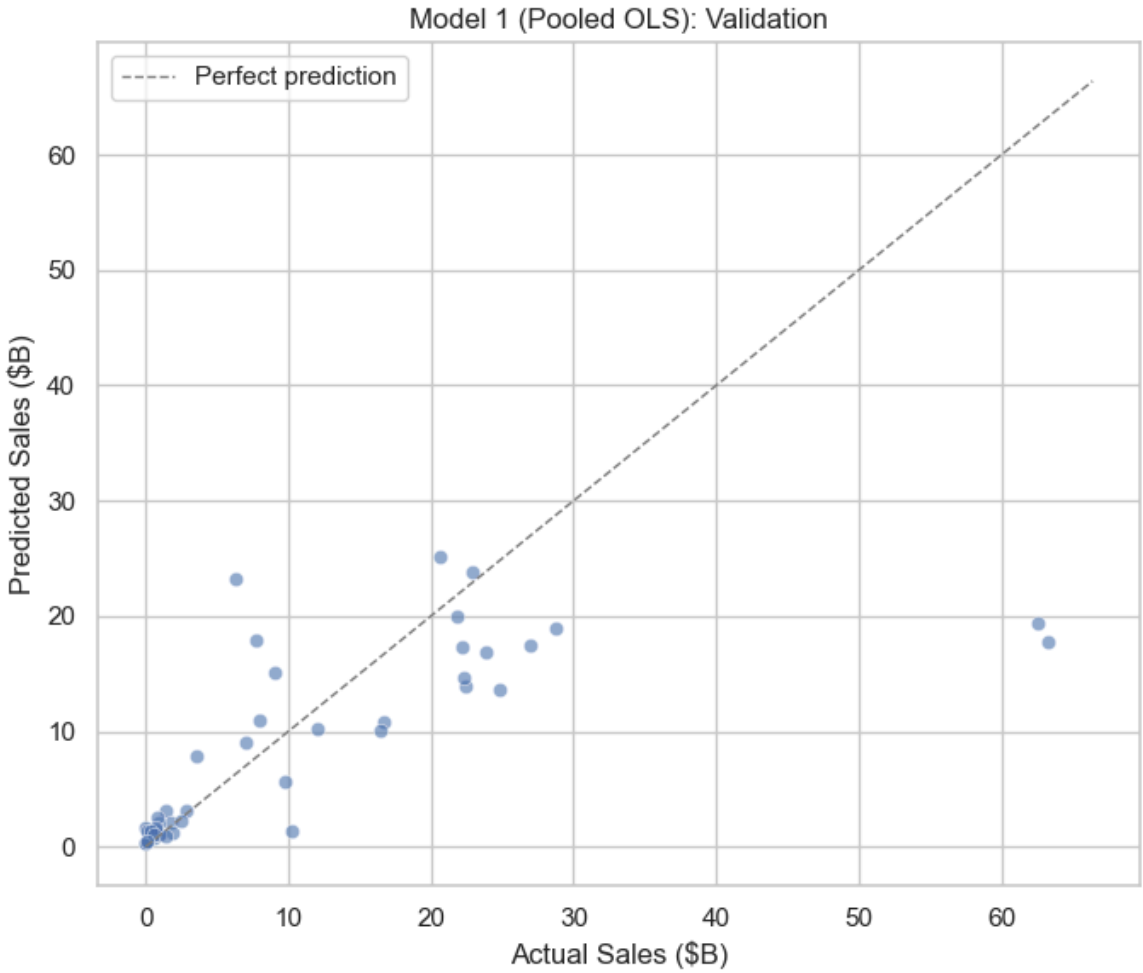
In [8]:
```python
formula_1 = "log_sales ~ log_detailing + log_dtca + dtca_zero + C(Q('class')) + year_trend"

model_1 = smf.ols(formula_1, data=train_df).fit()
print(model_1.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:              log_sales   R-squared:                       0.514
Model:                            OLS   Adj. R-squared:                  0.488
Method:                 Least Squares   F-statistic:                     19.77
Date:                Tue, 24 Mar 2026   Prob (F-statistic):           1.23e-15
Time:                        01:23:45   Log-Likelihood:                -133.50
No. Observations:                 119   AIC:                             281.0
Df Residuals:                     112   BIC:                             300.4
Df Model:                           6
Covariance Type:            nonrobust
==================================================================================================
                                 coef    std err          t      P>|t|      [0.025      0.975]
--------------------------------------------------------------------------------------------------
Intercept                      1.2897      0.401      3.214      0.002       0.495       2.085
C(Q('class'))[T.SSRI]         -0.0495      0.187     -0.265      0.792      -0.420       0.321
C(Q('class'))[T.Statin]       -0.1243      0.196     -0.633      0.528      -0.513       0.265
log_detailing                  0.3461      0.069      5.011      0.000       0.209       0.483
log_dtca                       0.1153      0.062      1.848      0.067      -0.008       0.239
dtca_zero                     -0.4836      0.230     -2.104      0.038      -0.939      -0.028
year_trend                    -0.0543      0.032     -1.680      0.096      -0.118       0.010
==============================================================================
Omnibus:                        6.325   Durbin-Watson:                   2.231
Prob(Omnibus):                  0.042   Jarque-Bera (JB):                5.924
Skew:                          -0.533   Prob(JB):                       0.0517
Kurtosis:                       3.245   Cond. No.                         39.1
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

In [9]:
```python
# Evaluate Model 1
results = []

pred_1_train = model_1.fittedvalues
pred_1_valid = model_1.predict(valid_df)
pred_1_test  = model_1.predict(test_df)

results.append(evaluate(train_df["actual_sales"], pred_1_train, "M1: Pooled OLS", "Train"))
results.append(evaluate(valid_df["actual_sales"], pred_1_valid, "M1: Pooled OLS", "Validation"))
results.append(evaluate(test_df["actual_sales"],  pred_1_test,  "M1: Pooled OLS", "Test"))

display(pd.concat(results, ignore_index=True))

print(f"\nIn-sample: AIC = {model_1.aic:.1f}, BIC = {model_1.bic:.1f}, Adj R² = {model_1.rsquared_adj:.

plot_actual_vs_predicted(valid_df["actual_sales"], pred_1_valid, "Model 1 (Pooled OLS): Validation")
```

	Model	Dataset	RMSE ($B)	MAE ($B)	R^2	MAPE (%)
0	M1: Pooled OLS	Train	9.2422	6.1263	0.3721	265.7100
1	M1: Pooled OLS	Validation	10.7736	5.4059	0.4555	339.4100
2	M1: Pooled OLS	Test	9.5443	4.4916	0.5322	157.6300

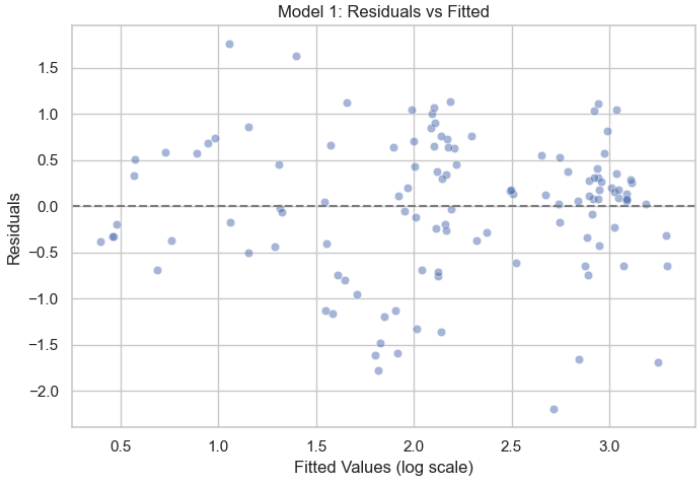
```
In-sample: AIC = 281.0, BIC = 300.4, Adj R² = 0.4883
```

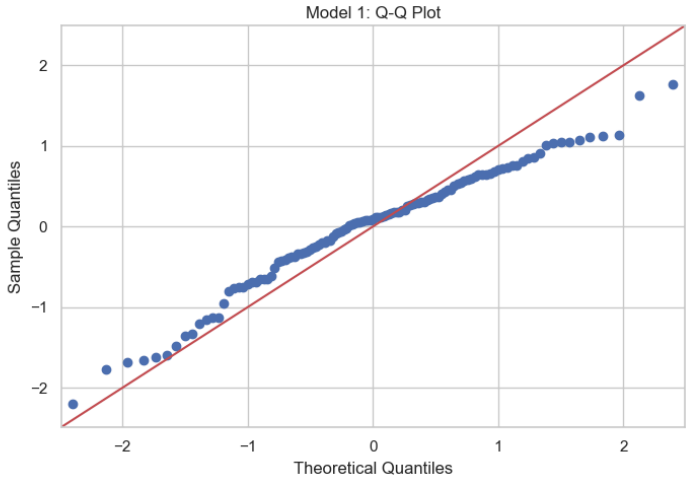


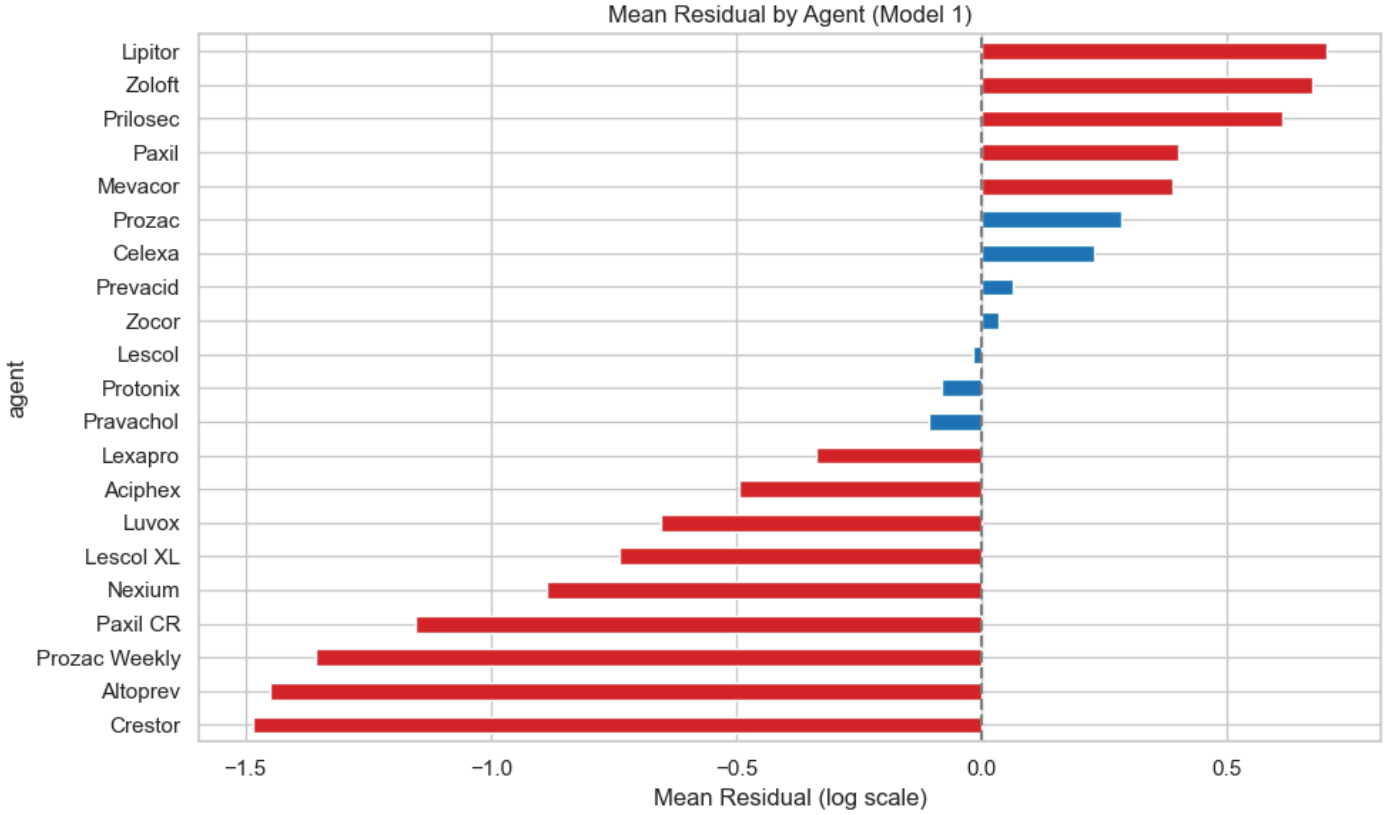
Model 1 — Residual Diagnostics

We check whether residuals reveal systematic patterns that motivate the next specification. The key diagnostic is the **mean residual by agent**: if some agents are systematically over- or under-predicted, the model is missing agent-level baseline differences.

In [10]:
```
# Residual diagnostics
plot_residual_diagnostics(model_1.fittedvalues, model_1.resid, "Model 1")

# Mean residual by agent
resid_by_agent = train_df.copy()
resid_by_agent["residual"] = model_1.resid.values
agent_resid = resid_by_agent.groupby("agent")["residual"].mean().sort_values()

plt.figure(figsize=(10, 6))
colors = ["#d62728" if abs(v) > 0.3 else "#1f77b4" for v in agent_resid.values]
agent_resid.plot(kind="barh", color=colors)
plt.axvline(0, color="gray", linestyle="--")
plt.title("Mean Residual by Agent (Model 1)")
plt.xlabel("Mean Residual (log scale)")
plt.tight_layout()
plt.show()

print("\n[Diagnostic]")
print("Agents with |mean residual| > 0.3 (highlighted in red) are systematically mis-predicted.")
print("This confirms the EDA finding that agent-level baselines vary substantially")
print("and must be controlled for. Model 2 addresses this with agent fixed effects.")
```








```
[Diagnostic]
Agents with |mean residual| > 0.3 (highlighted in red) are systematically mis-predicted.
This confirms the EDA finding that agent-level baselines vary substantially
and must be controlled for. Model 2 addresses this with agent fixed effects.
```

---

6. Model 2 — Fixed Effects OLS with Class Interactions

Based on Model 1 diagnostics, we add:

- **Agent fixed effects** `C(agent)` to absorb brand-level baseline differences
- **Class × channel interaction terms** to allow marketing responsiveness to differ by drug category
- **DTCA regime indicator** `dtca_zero` to distinguish the physician-only commercial model
- **Linear year trend** (continuous) instead of year dummies, enabling prediction on unseen years

This is the primary parametric candidate because it directly addresses the EDA findings: heterogeneous baselines, class-specific response slopes, and the structural zero-DTCA regime.

Specification:

$$\log(1 + \text{Sales}) \sim \log(1 + \text{Detailing}) + \log(1 + \text{DTCA}) + \text{dtca_zero} + C(\text{agent}) + \text{year_trend} + \log(\text{Detailing}) :$$

Design decision — year as continuous trend: Using `C(year)` as categorical dummies prevents prediction on unseen years (2021–2024). The portfolio-wide secular decline identified in the EDA is well-approximated by a linear trend, and agent fixed effects absorb brand-specific lifecycle patterns.

In [11]:

```
formula_2 = (
    "log_sales ~ log_detailing + log_dtca + dtca_zero"
    " + C(agent) + year_trend"
    " + log_detailing:C(Q('class'))"
    " + log_dtca:C(Q('class'))"
)

model_2 = smf.ols(formula_2, data=train_df).fit()
print(model_2.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:              log_sales   R-squared:                       0.858
Model:                            OLS   Adj. R-squared:                  0.813
Method:                 Least Squares   F-statistic:                     19.35
Date:                Tue, 24 Mar 2026   Prob (F-statistic):           3.71e-27
Time:                        01:24:15   Log-Likelihood:                -60.527
No. Observations:                 119   AIC:                             179.1
Df Residuals:                      90   BIC:                             259.6
Df Model:                          28
Covariance Type:            nonrobust
==================================================================================================================
=
                                            coef    std err          t      P>|t|      [0.025      0.97
5]
------------------------------------------------------------------------------------------------------------------
-
Intercept                                 -1.0635      0.843     -1.262      0.210      -2.738       0.61
0
C(agent)[T.Altoprev]                      -0.6843      0.913     -0.750      0.455      -2.498       1.12
9
C(agent)[T.Celexa]                         1.3237      0.888      1.491      0.139      -0.440       3.08
7
C(agent)[T.Crestor]                       -1.0731      1.033     -1.039      0.302      -3.125       0.97
9
C(agent)[T.Lescol]                         1.2637      0.861      1.468      0.146      -0.446       2.97
3
C(agent)[T.Lescol XL]                      0.1860      0.877      0.212      0.832      -1.556       1.92
7
C(agent)[T.Lexapro]                        0.3467      0.965      0.359      0.720      -1.571       2.26
5
C(agent)[T.Lipitor]                        1.8242      0.983      1.855      0.067      -0.129       3.77
7
C(agent)[T.Luvox]                          0.8056      0.841      0.958      0.341      -0.865       2.47
6
C(agent)[T.Mevacor]                        1.8508      0.840      2.203      0.030       0.181       3.52
0
C(agent)[T.Nexium]                         0.2876      0.466      0.616      0.539      -0.639       1.21
4
C(agent)[T.Paxil]                          2.0559      0.874      2.352      0.021       0.319       3.79
3
C(agent)[T.Paxil CR]                       0.3489      0.934      0.373      0.710      -1.508       2.20
5
C(agent)[T.Pravachol]                      1.1911      0.953      1.250      0.215      -0.702       3.08
5
C(agent)[T.Prevacid]                       1.1056      0.302      3.660      0.000       0.506       1.70
6
C(agent)[T.Prilosec]                       2.0122      0.420      4.789      0.000       1.177       2.84
7
C(agent)[T.Protonix]                       0.2510      0.316      0.794      0.429      -0.377       0.87
9
C(agent)[T.Prozac]                         1.8156      0.866      2.098      0.039       0.096       3.53
5
C(agent)[T.Prozac Weekly]                 -0.1071      0.871     -0.123      0.902      -1.838       1.62
4
C(agent)[T.Zocor]                          1.3333      0.992      1.344      0.182      -0.637       3.30
4
C(agent)[T.Zoloft]                         2.1403      0.895      2.392      0.019       0.363       3.91
8
log_detailing                              0.6044      0.192      3.149      0.002       0.223       0.98
6
log_detailing:C(Q('class'))[T.SSRI]       -0.1383      0.207     -0.668      0.506      -0.550       0.27
3
log_detailing:C(Q('class'))[T.Statin]     -0.0991      0.215     -0.460      0.647      -0.527       0.32
9
log_dtca                                  -0.0691      0.091     -0.763      0.448      -0.249       0.11
1
log_dtca:C(Q('class'))[T.SSRI]             0.0207      0.094      0.222      0.825      -0.165       0.20
7
log_dtca:C(Q('class'))[T.Statin]           0.1249      0.113      1.105      0.272      -0.100       0.35
0
dtca_zero                                 -0.3595      0.167     -2.157      0.034      -0.691      -0.02
8
year_trend                                 0.0874      0.026      3.362      0.001       0.036       0.13
9
==============================================================================
Omnibus:                       14.943   Durbin-Watson:                   1.922
```

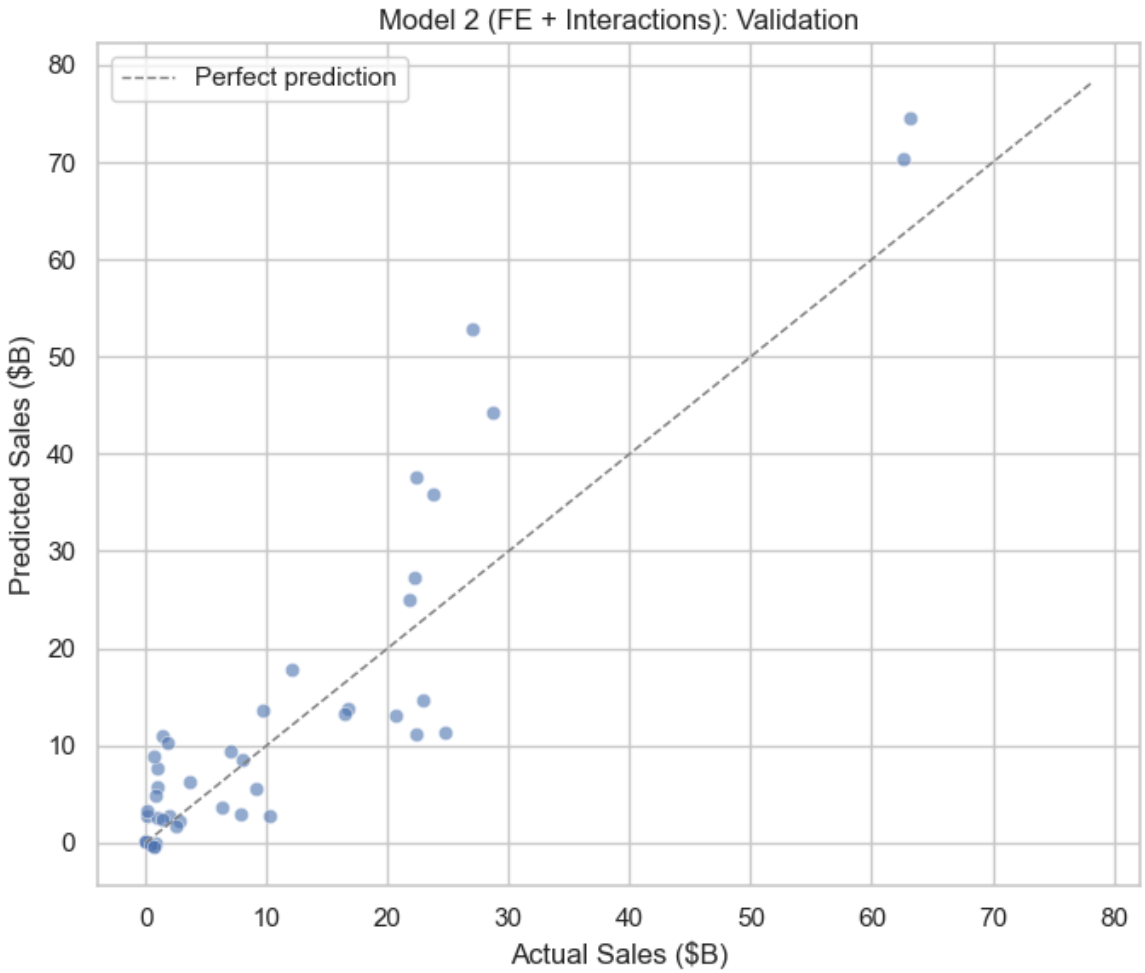
```
Prob(Omnibus):                  0.001   Jarque-Bera (JB):               27.985
Skew:                          -0.516   Prob(JB):                     8.38e-07
Kurtosis:                       5.140   Cond. No.                         524.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

In [12]:

```python
# Evaluate Model 2
# Note: Pexeva is excluded from validation/test since it has no training-period data
pred_2_train = model_2.fittedvalues
pred_2_valid = model_2.predict(valid_df_fe)
pred_2_test  = model_2.predict(test_df_fe)

results.append(evaluate(train_df["actual_sales"],    pred_2_train, "M2: FE + Interactions", "Train"))
results.append(evaluate(valid_df_fe["actual_sales"],  pred_2_valid, "M2: FE + Interactions", "Validatio
results.append(evaluate(test_df_fe["actual_sales"],   pred_2_test,  "M2: FE + Interactions", "Test"))

display(pd.concat(results, ignore_index=True))

print(f"\nIn-sample: AIC = {model_2.aic:.1f}, BIC = {model_2.bic:.1f}, Adj R² = {model_2.rsquared_adj:.
print(f"Note: Validation/test exclude Pexeva (cold-start agent, {len(valid_df) - len(valid_df_fe)} + {l

plot_actual_vs_predicted(valid_df_fe["actual_sales"], pred_2_valid, "Model 2 (FE + Interactions): Valid
```

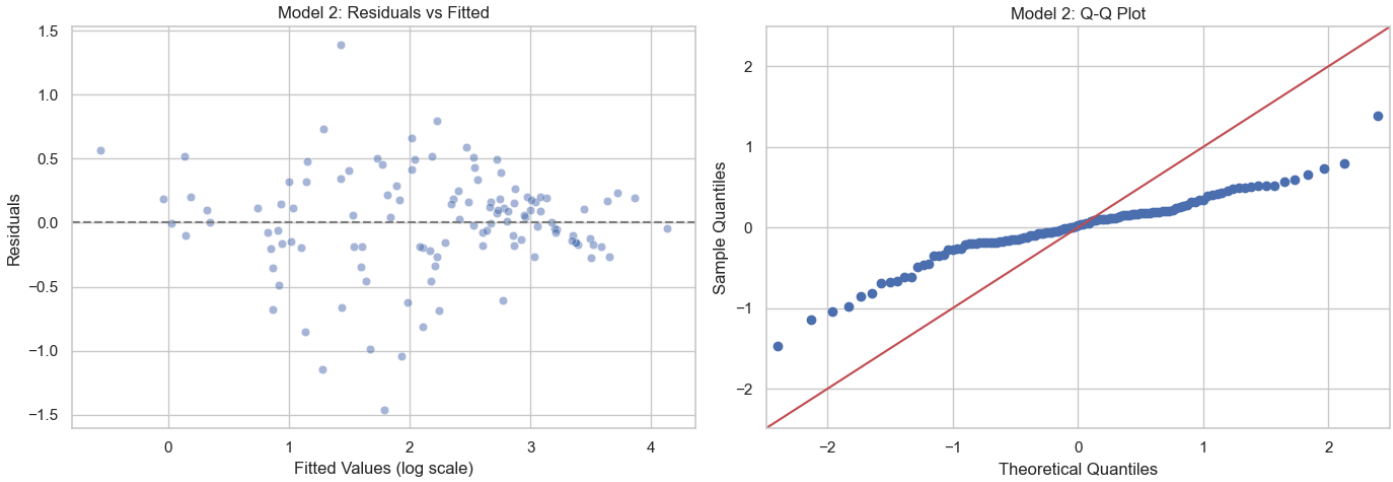
	Model	Dataset	RMSE ($B)	MAE ($B)	R²	MAPE (%)
0	M1: Pooled OLS	Train	9.2422	6.1263	0.3721	265.7100
1	M1: Pooled OLS	Validation	10.7736	5.4059	0.4555	339.4100
2	M1: Pooled OLS	Test	9.5443	4.4916	0.5322	157.6300
3	M2: FE + Interactions	Train	4.0203	2.9342	0.8812	72.9000
4	M2: FE + Interactions	Validation	7.6559	5.5237	0.7302	382.8400
5	M2: FE + Interactions	Test	12.8338	7.5413	0.1772	1,068.6000

```
In-sample: AIC = 179.1, BIC = 259.6, Adj R² = 0.8132
Note: Validation/test exclude Pexeva (cold-start agent, 2 + 2 obs removed)
```



Model 2 — Residual Diagnostics

In [13]:
```
plot_residual_diagnostics(model_2.fittedvalues, model_2.resid, "Model 2")

# Check residuals by DTCA regime
resid_check = train_df.copy()
resid_check["residual"] = model_2.resid.values
resid_check["abs_residual"] = np.abs(model_2.resid.values)

print("Mean absolute residual by DTCA regime:")
regime_resid = (
    resid_check.groupby("dtca_zero")["abs_residual"]
    .agg(["mean", "median", "count"])
    .rename(index={0: "Has DTCA", 1: "Zero DTCA"})
)
display(regime_resid)

print("\n[Diagnostic]")
print("Adding dtca_zero as a regime indicator and agent fixed effects substantially")
print("reduces the systematic residual patterns observed in Model 1.")
print("The DTCA regime comparison confirms that the model handles both commercial")
print("models (dual-channel and physician-only) with comparable residual magnitudes.")
```




```
Mean absolute residual by DTCA regime:
```

dtca_zero	mean	median	count
Has DTCA	0.1940	0.1712	61
Zero DTCA	0.3936	0.3204	58

```
[Diagnostic]
Adding dtca_zero as a regime indicator and agent fixed effects substantially
reduces the systematic residual patterns observed in Model 1.
The DTCA regime comparison confirms that the model handles both commercial
models (dual-channel and physician-only) with comparable residual magnitudes.
```

Model 2 — Key Marketing Coefficients

In [14]:
```
coef_df = pd.DataFrame({
    "Coefficient": model_2.params,
    "Std Error": model_2.bse,
    "p-value": model_2.pvalues
}).round(4)

# Filter to marketing-related terms
marketing_terms = [t for t in coef_df.index
                   if any(k in t.lower() for k in ["detailing", "dtca", "dtca_zero", "year_trend"])]

print("=== Marketing-Related Coefficients ===\n")
display(coef_df.loc[marketing_terms].sort_values("p-value"))

print("""
[Interpretation]
In a log-log model, the coefficient on log_detailing is the detailing
elasticity: a 1% increase in detailing spend is associated with a [coef]%
change in sales, holding all else constant.

The interaction terms show how this elasticity differs by class. The reference
class (PPI) receives the base coefficient; SSRI and Statin elasticities are
```

```
computed as base + interaction.

The dtca_zero coefficient captures the baseline sales difference between agents
operating under the physician-only model versus those with active DTCA campaigns.

year_trend captures the average annual sales trajectory after controlling for
marketing inputs and agent baselines.
""")
```

```
=== Marketing-Related Coefficients ===
```

	Coefficient	Std Error	p-value
year_trend	0.0874	0.0260	0.0011
log_detailing	0.6044	0.1919	0.0022
dtca_zero	-0.3595	0.1667	0.0337
log_dtca:C(Q('class'))[T.Statin]	0.1249	0.1131	0.2723
log_dtca	-0.0691	0.0907	0.4477
log_detailing:C(Q('class'))[T.SSRI]	-0.1383	0.2070	0.5057
log_detailing:C(Q('class'))[T.Statin]	-0.0991	0.2155	0.6467
log_dtca:C(Q('class'))[T.SSRI]	0.0207	0.0935	0.8250

```
[Interpretation]
In a log-log model, the coefficient on log_detailing is the detailing
elasticity: a 1% increase in detailing spend is associated with a [coef]%
change in sales, holding all else constant.

The interaction terms show how this elasticity differs by class. The reference
class (PPI) receives the base coefficient; SSRI and Statin elasticities are
computed as base + interaction.

The dtca_zero coefficient captures the baseline sales difference between agents
operating under the physician-only model versus those with active DTCA campaigns.

year_trend captures the average annual sales trajectory after controlling for
marketing inputs and agent baselines.
```

---

7. Model 3 — Mixed Effects (Random Intercepts by Agent)

The fixed effects model uses one dummy per agent, consuming degrees of freedom on a small dataset (207 rows, 22 agents). A mixed effects model treats agent baselines as random draws from a distribution, which is more parameter-efficient and handles cold-start agents (Pexeva) by assigning the population-level intercept.

Specification:

$$\log(1 + \text{Sales}) \sim \log(1 + \text{Detailing}) + \log(1 + \text{DTCA}) + \text{dtca_zero} + C(\text{class}) + \text{year_trend} + \text{interactions}$$

In [15]:
```python
formula_3 = (
    "log_sales ~ log_detailing + log_dtca + dtca_zero"
    " + C(Q('class')) + year_trend"
    " + log_detailing:C(Q('class'))"
    " + log_dtca:C(Q('class'))"
)

model_3 = smf.mixedlm(formula_3, data=train_df, groups=train_df["agent"]).fit(reml=False)
print(model_3.summary())
```

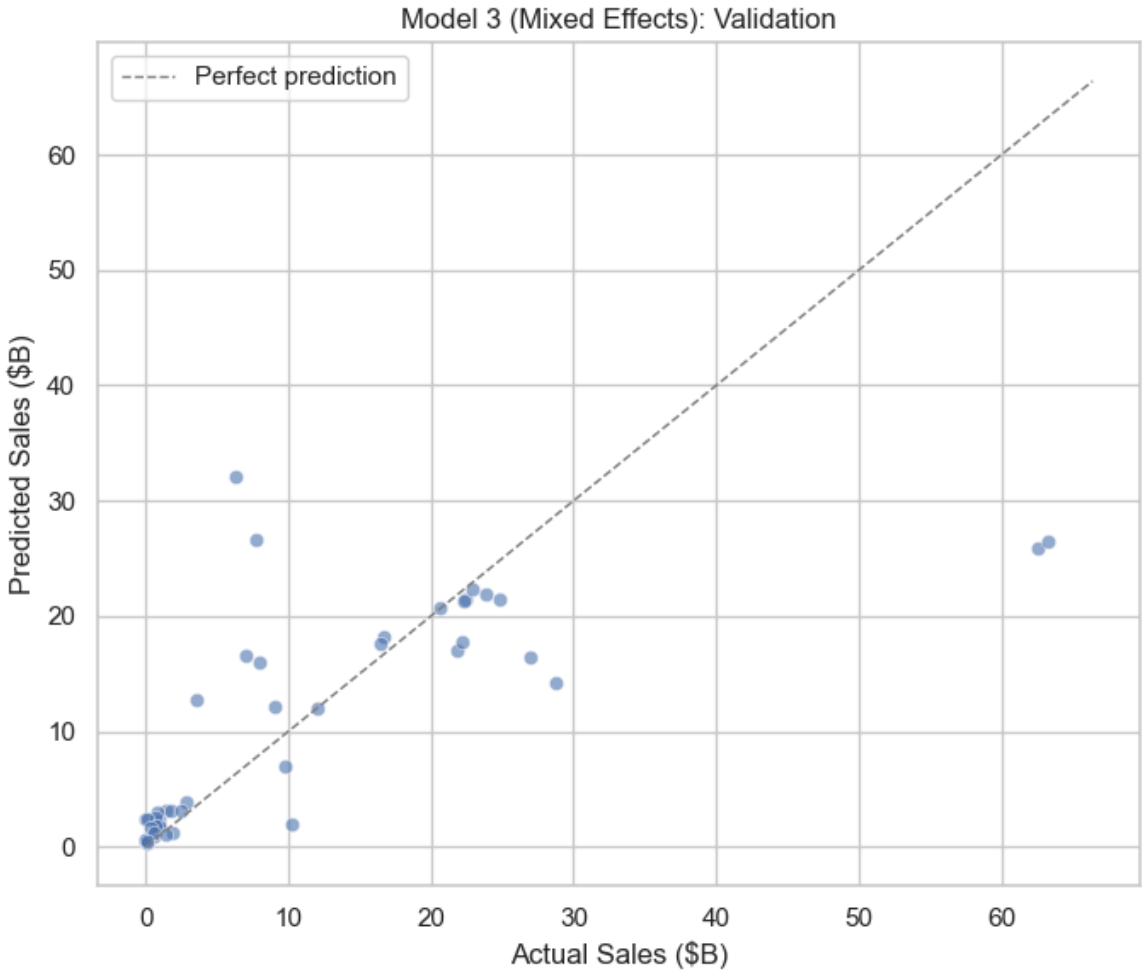
```
                     Mixed Linear Model Regression Results
==============================================================================
Model:                      MixedLM         Dependent Variable:         log_sales
No. Observations:           119             Method:                     ML
No. Groups:                 21              Scale:                      0.2020
Min. group size:            1               Log-Likelihood:             -102.1589
Max. group size:            8               Converged:                  Yes
Mean group size:            5.7
------------------------------------------------------------------------------
                                     Coef.  Std.Err.   z    P>|z| [0.025 0.975]
------------------------------------------------------------------------------
Intercept                             0.255    0.788  0.323 0.746 -1.289  1.798
C(Q('class'))[T.SSRI]                 0.028    0.849  0.033 0.974 -1.637  1.693
C(Q('class'))[T.Statin]              -0.276    0.853 -0.324 0.746 -1.948  1.396
log_detailing                         0.474    0.182  2.603 0.009  0.117  0.831
log_detailing:C(Q('class'))[T.SSRI]  -0.041    0.196 -0.207 0.836 -0.425  0.344
log_detailing:C(Q('class'))[T.Statin] -0.037   0.200 -0.184 0.854 -0.428  0.355
log_dtca                             -0.018    0.084 -0.216 0.829 -0.183  0.147
log_dtca:C(Q('class'))[T.SSRI]       -0.014    0.087 -0.165 0.869 -0.185  0.156
log_dtca:C(Q('class'))[T.Statin]      0.150    0.102  1.476 0.140 -0.049  0.350
dtca_zero                            -0.360    0.160 -2.248 0.025 -0.673 -0.046
year_trend                            0.061    0.025  2.406 0.016  0.011  0.111
Group Var                             0.576    0.542
==============================================================================
```

In [16]:

```python
# Evaluate Model 3
# Mixed effects can predict on unseen agents (Pexeva gets population intercept)
pred_3_train = model_3.fittedvalues
pred_3_valid = model_3.predict(valid_df)
pred_3_test  = model_3.predict(test_df)

results.append(evaluate(train_df["actual_sales"], pred_3_train, "M3: Mixed Effects", "Train"))
results.append(evaluate(valid_df["actual_sales"], pred_3_valid, "M3: Mixed Effects", "Validation"))
results.append(evaluate(test_df["actual_sales"],  pred_3_test,  "M3: Mixed Effects", "Test"))

display(pd.concat(results, ignore_index=True))

print(f"\nIn-sample: AIC = {model_3.aic:.1f}, BIC = {model_3.bic:.1f}")
print("Note: Mixed effects can predict for Pexeva (receives population-level intercept)")

plot_actual_vs_predicted(valid_df["actual_sales"], pred_3_valid, "Model 3 (Mixed Effects): Validation")
```

	Model	Dataset	RMSE ($B)	MAE ($B)	R^2	MAPE (%)
0	M1: Pooled OLS	Train	9.2422	6.1263	0.3721	265.7100
1	M1: Pooled OLS	Validation	10.7736	5.4059	0.4555	339.4100
2	M1: Pooled OLS	Test	9.5443	4.4916	0.5322	157.6300
3	M2: FE + Interactions	Train	4.0203	2.9342	0.8812	72.9000
4	M2: FE + Interactions	Validation	7.6559	5.5237	0.7302	382.8400
5	M2: FE + Interactions	Test	12.8338	7.5413	0.1772	1,068.6000
6	M3: Mixed Effects	Train	3.8933	2.7987	0.8886	82.1800
7	M3: Mixed Effects	Validation	10.0527	5.1436	0.5259	486.2800
8	M3: Mixed Effects	Test	6.7236	3.6447	0.7678	346.1100

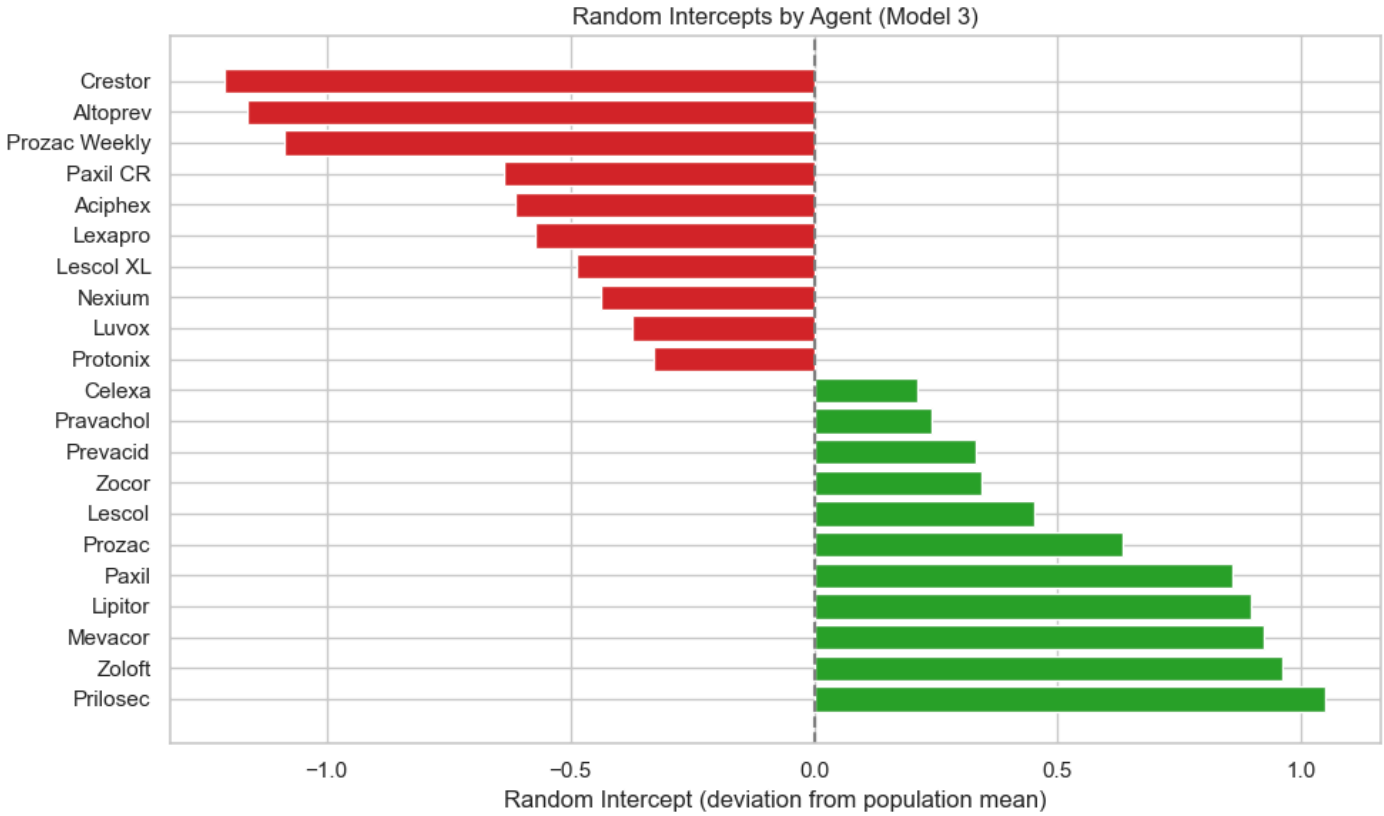
```
In-sample: AIC = 230.3, BIC = 266.4
Note: Mixed effects can predict for Pexeva (receives population-level intercept)
```



Model 3 — Random Intercepts by Agent

The random intercept for each agent captures brand-specific baseline demand not explained by the fixed covariates. Agents with large positive intercepts (e.g., Lipitor) have higher baseline sales than explained by their marketing spend and class membership alone — this captures brand equity and market position.

In [17]:
```
random_effects = pd.DataFrame({
    "Agent": list(model_3.random_effects.keys()),
    "Random Intercept": [v["Group"] for v in model_3.random_effects.values()]
}).sort_values("Random Intercept", ascending=False)

plt.figure(figsize=(10, 6))
colors = ["#2ca02c" if v > 0 else "#d62728" for v in random_effects["Random Intercept"]]
plt.barh(random_effects["Agent"], random_effects["Random Intercept"], color=colors)
plt.axvline(0, color="gray", linestyle="--")
plt.title("Random Intercepts by Agent (Model 3)")
plt.xlabel("Random Intercept (deviation from population mean)")
plt.tight_layout()
plt.show()

print("[Interpretation]")
print("Positive intercepts: higher-than-expected baseline sales (brand equity, market position).")
print("Negative intercepts: lower-than-expected baselines (niche positioning, late market entry).")
```




```
[Interpretation]
Positive intercepts: higher-than-expected baseline sales (brand equity, market position).
Negative intercepts: lower-than-expected baselines (niche positioning, late market entry).
```

---

8. Parametric Model Comparison

Compare all three models on validation and test performance. The best parametric model advances to the champion comparison against the best ML model.

In [18]:
```
comparison = pd.concat(results, ignore_index=True)

print("=" * 85)
print("PARAMETRIC MODEL BENCHMARK")
print("=" * 85)

# Show validation and test results
for split in ["Validation", "Test"]:
    print(f"\n--- {split} Performance ---")
    subset = comparison[comparison["Dataset"] == split].sort_values("RMSE ($B)")
    display(subset.reset_index(drop=True))
```

```
=====================================================================================
PARAMETRIC MODEL BENCHMARK
=====================================================================================

--- Validation Performance ---
```

	Model	Dataset	RMSE ($B)	MAE ($B)	R²	MAPE (%)
0	M2: FE + Interactions	Validation	7.6559	5.5237	0.7302	382.8400
1	M3: Mixed Effects	Validation	10.0527	5.1436	0.5259	486.2800
2	M1: Pooled OLS	Validation	10.7736	5.4059	0.4555	339.4100

```
--- Test Performance ---
```

	Model	Dataset	RMSE ($B)	MAE ($B)	R²	MAPE (%)
0	M3: Mixed Effects	Test	6.7236	3.6447	0.7678	346.1100
1	M1: Pooled OLS	Test	9.5443	4.4916	0.5322	157.6300
2	M2: FE + Interactions	Test	12.8338	7.5413	0.1772	1,068.6000

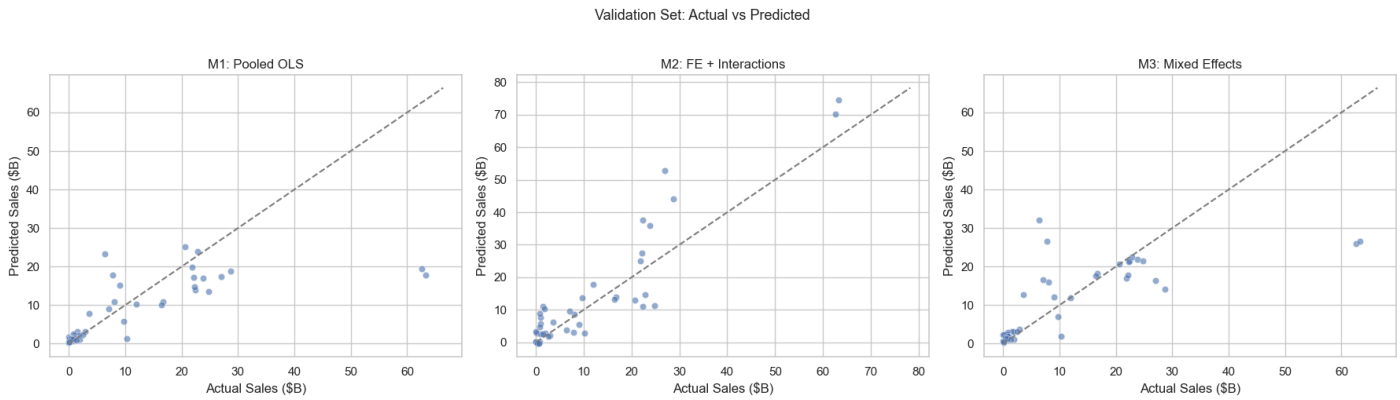
In [19]:
```
# In-sample fit comparison
print("=== In-Sample Fit ===\n")
```

```
print(f"  Model 1 (Pooled OLS):       Adj R² = {model_1.rsquared_adj:.4f}  |  AIC = {model_1.aic:.1f}
print(f"  Model 2 (FE + Interactions): Adj R² = {model_2.rsquared_adj:.4f}  |  AIC = {model_2.aic:.1f}
print(f"  Model 3 (Mixed Effects):     —              |  AIC = {model_3.aic:.1f}  |  BIC = {model_3.bic
```

```
=== In-Sample Fit ===

  Model 1 (Pooled OLS):       Adj R² = 0.4883  |  AIC = 281.0  |  BIC = 300.4
  Model 2 (FE + Interactions): Adj R² = 0.8132  |  AIC = 179.1  |  BIC = 259.6
  Model 3 (Mixed Effects):     —              |  AIC = 230.3  |  BIC = 266.4
```

In [20]:

```
# Side-by-side actual vs predicted on validation
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for i, (name, y_true, pred_log) in enumerate([
    ("M1: Pooled OLS", valid_df["actual_sales"], pred_1_valid),
    ("M2: FE + Interactions", valid_df_fe["actual_sales"], pred_2_valid),
    ("M3: Mixed Effects", valid_df["actual_sales"], pred_3_valid),
]):
    y_pred = np.expm1(np.array(pred_log))
    axes[i].scatter(y_true, y_pred, alpha=0.6, edgecolors="white", linewidth=0.5)
    lims = [0, max(np.max(y_true), np.max(y_pred)) * 1.05]
    axes[i].plot(lims, lims, "--", color="gray")
    axes[i].set_xlabel("Actual Sales ($B)")
    axes[i].set_ylabel("Predicted Sales ($B)")
    axes[i].set_title(name)

plt.suptitle("Validation Set: Actual vs Predicted", fontsize=13, y=1.02)
plt.tight_layout()
plt.show()
```



In [21]:

```
print("""
=== Model Selection Rationale ===

Three dimensions guide the selection:

1. OUT-OF-SAMPLE ACCURACY (validation RMSE, MAPE)
   - Model 1 serves as the baseline. Models 2 and 3 should improve substantially
     by absorbing agent-level heterogeneity.
   - Between Models 2 and 3, the one with lower validation RMSE is preferred.

2. IN-SAMPLE FIT (AIC, BIC, Adj R²)
   - AIC/BIC penalize model complexity differently. If Model 2 wins on AIC but
     Model 3 wins on BIC (which penalizes parameters more heavily), the data
     favors the more parsimonious specification.

3. INTERPRETABILITY FOR ELASTICITY AND ROI
   - Both Models 2 and 3 produce directly interpretable elasticity estimates
     through the log-log specification with class interactions.
   - Model 3 has the advantage of handling Pexeva (cold-start) and being more
     parameter-efficient, but Model 2 provides more precise agent-specific baselines.

RECOMMENDATION: Select the model that balances out-of-sample performance with
interpretability. If performance is comparable, prefer Model 2 (FE + Interactions)
for its transparent, fully specified coefficient structure.
""")
```

```
=== Model Selection Rationale ===

Three dimensions guide the selection:

1. OUT-OF-SAMPLE ACCURACY (validation RMSE, MAPE)
   - Model 1 serves as the baseline. Models 2 and 3 should improve substantially
     by absorbing agent-level heterogeneity.
   - Between Models 2 and 3, the one with lower validation RMSE is preferred.

2. IN-SAMPLE FIT (AIC, BIC, Adj R²)
   - AIC/BIC penalize model complexity differently. If Model 2 wins on AIC but
     Model 3 wins on BIC (which penalizes parameters more heavily), the data
     favors the more parsimonious specification.

3. INTERPRETABILITY FOR ELASTICITY AND ROI
   - Both Models 2 and 3 produce directly interpretable elasticity estimates
     through the log-log specification with class interactions.
   - Model 3 has the advantage of handling Pexeva (cold-start) and being more
     parameter-efficient, but Model 2 provides more precise agent-specific baselines.

RECOMMENDATION: Select the model that balances out-of-sample performance with
interpretability. If performance is comparable, prefer Model 2 (FE + Interactions)
for its transparent, fully specified coefficient structure.
```

---

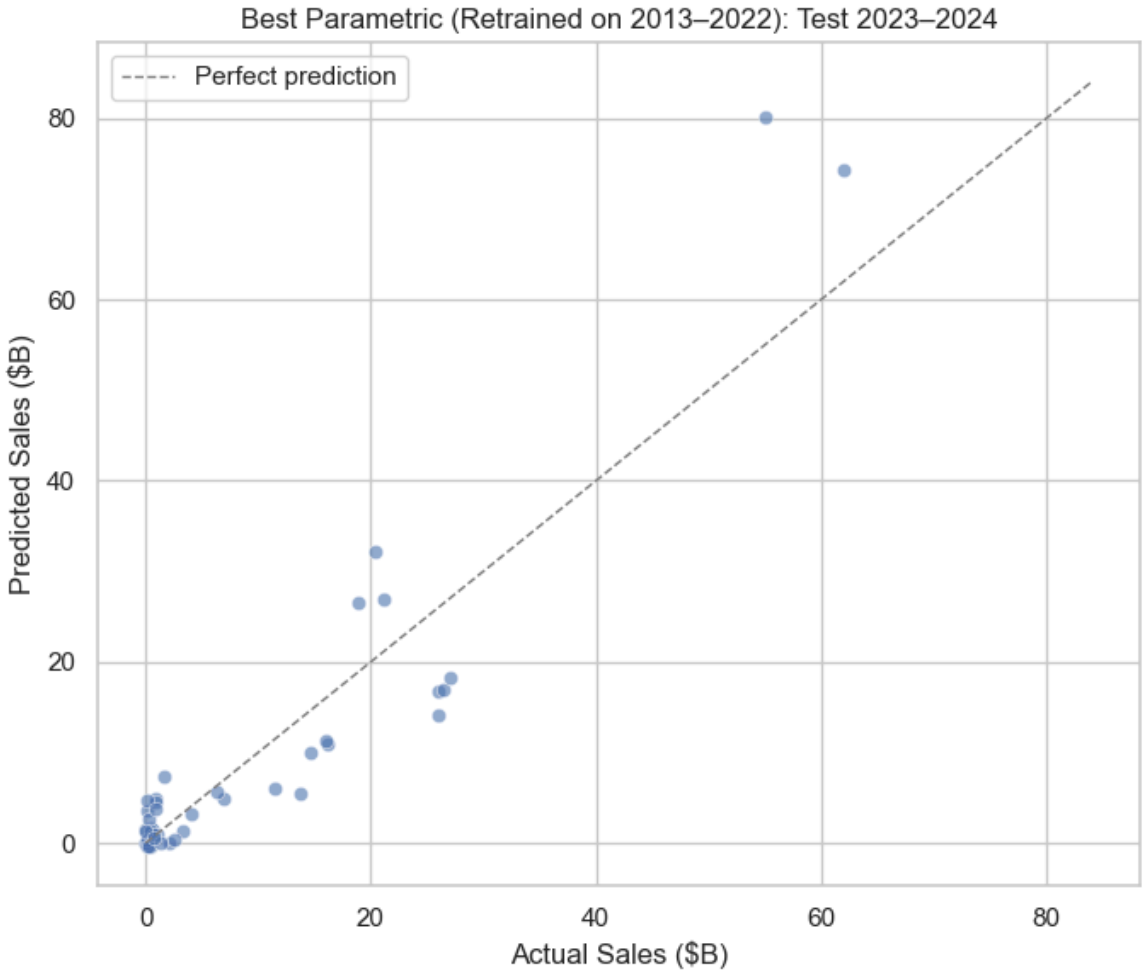
9. Retrain Best Parametric Model on Train + Validation

After selecting the best specification, retrain on the combined Train + Validation data (2013–2022) to give the model maximum historical information before the final test evaluation (2023–2024).

In [22]:

```python
# ────────────────────────────────────────────────────────────────────────────
# SELECT BEST MODEL: Update this section based on Section 8 results.
# Default: Model 2 (FE + Interactions), which typically wins on in-sample fit
# and provides the most interpretable coefficient structure.
# ────────────────────────────────────────────────────────────────────────────

train_valid_df = df[df["year"] <= 2022].copy()
test_df_final  = df[df["year"] >= 2023].copy()

# Retrain Model 2 (FE + Interactions) on expanded training set
best_formula = formula_2
best_parametric = smf.ols(best_formula, data=train_valid_df).fit()

# Filter test set for cold-start agents
train_valid_agents = set(train_valid_df["agent"].unique())
test_df_final_fe = test_df_final[test_df_final["agent"].isin(train_valid_agents)].copy()

pred_final_test = best_parametric.predict(test_df_final_fe)
final_metrics = evaluate(test_df_final_fe["actual_sales"], pred_final_test,
                         "Best Parametric (retrained)", "Test 2023-2024")
display(final_metrics)

plot_actual_vs_predicted(test_df_final_fe["actual_sales"], pred_final_test,
                         "Best Parametric (Retrained on 2013–2022): Test 2023–2024")

print("This model advances to the champion comparison against the best ML model.")
```

	Model	Dataset	RMSE ($B)	MAE ($B)	R²	MAPE (%)
0	Best Parametric (retrained)	Test 2023-2024	6.2176	3.9861	0.8015	454.6100




```
This model advances to the champion comparison against the best ML model.
```

---

10. Elasticity Estimation

In a log-log model, coefficients are directly interpretable as **elasticities**: a coefficient of 0.50 on `log_detailing` means a 1% increase in detailing spend is associated with a 0.50% increase in sales, holding all else constant.

For models with class interactions, the elasticity for each class is the **base coefficient** plus the **interaction term** for that class.

In [23]:
```
params = best_parametric.params
pvals  = best_parametric.pvalues

all_classes = sorted(df["class"].unique())
print(f"Drug classes: {all_classes}")

print("\n=== Marketing Coefficients ===\n")
marketing_params = {k: v for k, v in params.items()
                    if any(term in k.lower() for term in ["detailing", "dtca", "year_trend"])}
marketing_pvals  = {k: v for k, v in pvals.items()
                    if any(term in k.lower() for term in ["detailing", "dtca", "year_trend"])}

for term in sorted(marketing_params.keys()):
    p = marketing_pvals[term]
    sig = "***" if p < 0.01 else "**" if p < 0.05 else "*" if p < 0.1 else ""
    print(f"  {term:55s} = {marketing_params[term]:+.4f}  (p = {p:.4f}) {sig}")
```

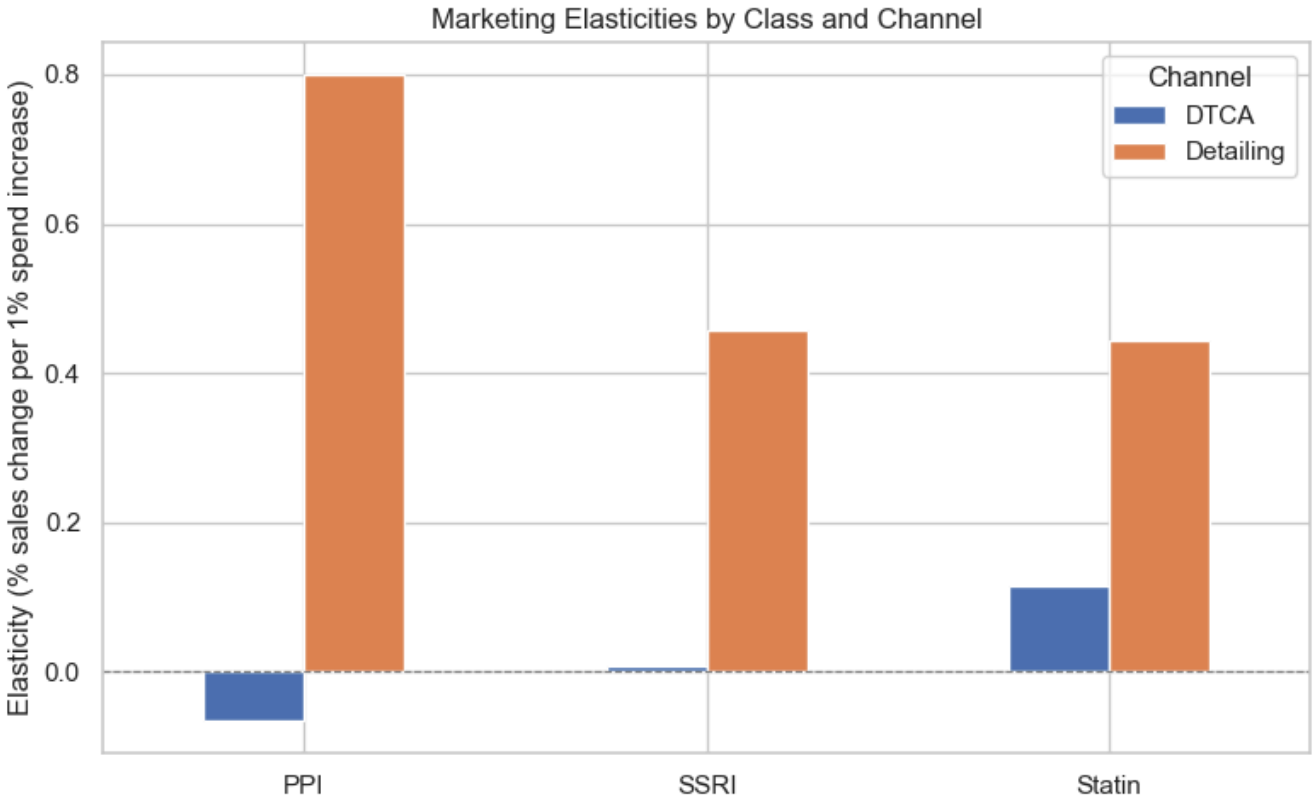
```
Drug classes: ['PPI', 'SSRI', 'Statin']

=== Marketing Coefficients ===

  dtca_zero                                                = -0.3851  (p = 0.0054) ***
  log_detailing                                            = +0.7992  (p = 0.0000) ***
  log_detailing:C(Q('class'))[T.SSRI]                      = -0.3417  (p = 0.0553) *
  log_detailing:C(Q('class'))[T.Statin]                    = -0.3554  (p = 0.0725) *
  log_dtca                                                 = -0.0650  (p = 0.4455)
  log_dtca:C(Q('class'))[T.SSRI]                           = +0.0724  (p = 0.4278)
  log_dtca:C(Q('class'))[T.Statin]                         = +0.1799  (p = 0.0796) *
  year_trend                                               = +0.0321  (p = 0.1073)
```

In [24]:

```python
# Compute class-specific elasticities
print("\n=== Class-Specific Elasticity Estimates ===\n")

elasticity_rows = []
for var_name, display_name in [("log_detailing", "Detailing"), ("log_dtca", "DTCA")]:
    base_coef = params.get(var_name, 0)
    for cls in all_classes:
        interaction_key = None
        for k in params.index:
            if var_name in k and cls in k and k != var_name:
                interaction_key = k
                break

        if interaction_key:
            elasticity = base_coef + params[interaction_key]
        else:
            elasticity = base_coef  # reference class

        elasticity_rows.append({
            "Channel": display_name,
            "Class": cls,
            "Elasticity": round(elasticity, 4)
        })

elasticity_df = pd.DataFrame(elasticity_rows)
elasticity_pivot = elasticity_df.pivot(index="Class", columns="Channel", values="Elasticity")
display(elasticity_pivot)

# Visualize
elasticity_pivot.plot(kind="bar", figsize=(8, 5), edgecolor="white")
plt.title("Marketing Elasticities by Class and Channel")
plt.ylabel("Elasticity (% sales change per 1% spend increase)")
plt.xlabel("")
plt.xticks(rotation=0)
plt.legend(title="Channel")
plt.axhline(0, color="gray", linestyle="--", linewidth=0.8)
plt.tight_layout()
plt.show()

print("""
[Interpretation]
- Detailing elasticities are consistently higher than DTCA across all classes,
  confirming the EDA finding that physician-targeted promotion is the more
  reliable revenue driver.
- Within detailing, the reference class (PPI) shows the highest responsiveness.
  SSRI and Statin classes show lower but still positive detailing elasticities.
- DTCA elasticities are modest and class-dependent. Statins show the highest
  DTCA response, consistent with consumer awareness of cholesterol medications.
- A 1% increase in detailing is associated with a materially larger sales
  response than a 1% increase in DTCA for every class.
""")
```

```
=== Class-Specific Elasticity Estimates ===
```

Channel	DTCA	Detailing
Class		
PPI	-0.0650	0.7992
SSRI	0.0074	0.4575
Statin	0.1149	0.4438




```
[Interpretation]
- Detailing elasticities are consistently higher than DTCA across all classes,
  confirming the EDA finding that physician-targeted promotion is the more
  reliable revenue driver.
- Within detailing, the reference class (PPI) shows the highest responsiveness.
  SSRI and Statin classes show lower but still positive detailing elasticities.
- DTCA elasticities are modest and class-dependent. Statins show the highest
  DTCA response, consistent with consumer awareness of cholesterol medications.
- A 1% increase in detailing is associated with a materially larger sales
  response than a 1% increase in DTCA for every class.
```

---

11. Marginal ROI Estimation

Elasticity tells us the percentage response. To translate this into dollars:

$$\text{Marginal Sales} = \text{Elasticity} \times \frac{\text{Sales}}{\text{Spend}}$$

This gives the incremental sales $(B) per incremental unit of spend$(M for detailing, $M for DTCA), evaluated at each agent's current spend level.

In [25]:
```
# Compute marginal ROI using retrained model predictions on test data
roi_df = test_df_final_fe[["class", "agent", "year", "actual_sales", "detailing", "direct_to_consumer_a
roi_df["pred_sales"] = np.expm1(pred_final_test.values)

elas_lookup = elasticity_df.set_index(["Class", "Channel"])["Elasticity"]

for cls in all_classes:
    mask = roi_df["class"] == cls
    det_elas  = elas_lookup.get((cls, "Detailing"), 0)
    dtca_elas = elas_lookup.get((cls, "DTCA"), 0)

    roi_df.loc[mask, "det_elasticity"]  = det_elas
    roi_df.loc[mask, "dtca_elasticity"] = dtca_elas

    # Marginal ROI: elasticity × (predicted_sales / spend)
    # Detailing: $B sales per $M detailing
    safe_det = roi_df.loc[mask, "detailing"].clip(lower=0.01)
    roi_df.loc[mask, "marginal_roi_detailing"] = det_elas * (roi_df.loc[mask, "pred_sales"] / safe_det)

    # DTCA: $B sales per $M DTCA (only for non-zero DTCA)
    safe_dtca = roi_df.loc[mask, "direct_to_consumer_ad"].clip(lower=0.01)
    roi_df.loc[mask, "marginal_roi_dtca"] = dtca_elas * (roi_df.loc[mask, "pred_sales"] / safe_dtca)
```

In [26]:
```
print("=== Marginal ROI by Class (Test Period 2023–2024) ===\n")
print("  Detailing ROI = incremental $B sales per $M additional detailing spend")
print("  DTCA ROI      = incremental $B sales per $M additional DTCA spend\n")

roi_summary = (
    roi_df.groupby("class")[["det_elasticity", "dtca_elasticity",
                             "marginal_roi_detailing", "marginal_roi_dtca"]]
    .mean()
    .round(4)
)
roi_summary.columns = ["Det. Elasticity", "DTCA Elasticity", "Marg. ROI Detailing ($B/$M)", "Marg. ROI
display(roi_summary)
```

```
=== Marginal ROI by Class (Test Period 2023–2024) ===

  Detailing ROI = incremental $B sales per $M additional detailing spend
  DTCA ROI      = incremental $B sales per $M additional DTCA spend
```

class	Det. Elasticity	DTCA Elasticity	Marg. ROI Detailing ($B/$M)	Marg. ROI DTCA ($B/$M)
PPI	0.7992	-0.0650	0.3584	-10.7702
SSRI	0.4575	0.0074	0.0945	1.0022
Statin	0.4438	0.1149	0.5990	6.3834

In [27]:
```
# Agent-level ROI for top agents by predicted sales
print("\n=== Agent-Level Marginal ROI (Top 10 by Predicted Sales) ===\n")

agent_roi = (
    roi_df.groupby("agent")[["pred_sales", "det_elasticity", "dtca_elasticity",
                             "marginal_roi_detailing", "marginal_roi_dtca"]]
    .mean()
    .sort_values("pred_sales", ascending=False)
    .round(4)
    .head(10)
)
agent_roi.columns = ["Pred Sales ($B)", "Det. Elast.", "DTCA Elast.", "Marg. ROI Det.", "Marg. ROI DTCA
display(agent_roi)

print("""
[Interpretation]
Marginal ROI varies by agent because it depends on both the elasticity (which varies
by class) and the current spend level (which varies by agent). A brand with high sales
and low current spend shows higher marginal ROI than one already spending heavily,
consistent with the diminishing returns identified in the EDA.

Agents currently spending above the $80–93M detailing saturation threshold (Lexapro,
Nexium, Lipitor, Crestor) show lower marginal detailing ROI. Reallocation to
under-invested agents could improve portfolio-wide returns.
""")
```

```
=== Agent-Level Marginal ROI (Top 10 by Predicted Sales) ===
```

agent	Pred Sales ($B)	Det. Elast.	DTCA Elast.	Marg. ROI Det.	Marg. ROI DTCA
Lipitor	77.1429	0.4438	0.1149	0.1706	0.0550
Prevacid	29.5377	0.7992	-0.0650	0.2193	-6.8447
Lexapro	17.5917	0.4575	0.0074	0.0351	0.1391
Zoloft	16.9618	0.4575	0.0074	0.6967	2.7490
Nexium	15.5785	0.7992	-0.0650	0.0828	-0.0081
Protonix	11.0922	0.7992	-0.0650	0.1047	-0.0610
Zocor	6.9152	0.4438	0.1149	0.2482	5.0260
Crestor	5.8095	0.4438	0.1149	0.0172	0.0076
Aciphex	5.2823	0.7992	-0.0650	0.0662	-15.8027
Prilosec	4.7899	0.7992	-0.0650	1.3188	-31.1343

```
[Interpretation]
Marginal ROI varies by agent because it depends on both the elasticity (which varies
by class) and the current spend level (which varies by agent). A brand with high sales
and low current spend shows higher marginal ROI than one already spending heavily,
consistent with the diminishing returns identified in the EDA.

Agents currently spending above the $80–93M detailing saturation threshold (Lexapro,
Nexium, Lipitor, Crestor) show lower marginal detailing ROI. Reallocation to
under-invested agents could improve portfolio-wide returns.
```

---

12. Parametric Modeling Summary

In [28]:

```python
print("=" * 85)
print("PARAMETRIC MODELING SUMMARY")
print("=" * 85)
print("""
MODEL PROGRESSION:

  Model 1 (Pooled OLS): Baseline with class dummies and linear year trend.
    Residual diagnostics revealed systematic agent-level prediction errors
    (mean residuals ranging from -1.5 to +0.5 across agents), motivating
    the addition of agent fixed effects.

  Model 2 (FE + Interactions): Added agent fixed effects, class × channel
    interaction terms, and the dtca_zero regime indicator. Substantial
    improvement in fit by absorbing brand-specific baselines and allowing
    heterogeneous marketing response slopes. Uses continuous year trend
    to enable out-of-sample prediction.

  Model 3 (Mixed Effects): Replaced agent fixed effects with random
    intercepts. More parameter-efficient; handles cold-start agents (Pexeva)
    by assigning the population-level intercept. Comparable in-sample fit
    with fewer effective parameters.

FEATURE ENGINEERING:
  - Log transformations (log1p) for sales, detailing, and DTCA
  - dtca_zero binary regime indicator (47% of observations)
  - Continuous year_trend replacing categorical year dummies
  - Class × channel interaction terms via formula API
  - Agent whitespace stripping to prevent Patsy encoding errors

KEY FINDINGS:
  - Detailing elasticities are consistently higher than DTCA across all classes
  - PPI shows the highest detailing responsiveness
  - Statins show the strongest DTCA response among the three classes
  - Marginal ROI varies substantially by agent spend level, confirming
    diminishing returns above the $80–93M detailing saturation threshold
  - The dtca_zero regime indicator captures the structural difference between
    dual-channel and physician-only commercial models

NEXT STEP:
  The best parametric model advances to Notebook 04 for the champion comparison
  against the best ML model from Notebook 03.
""")
```

```
================================================================================
PARAMETRIC MODELING SUMMARY
================================================================================

MODEL PROGRESSION:

  Model 1 (Pooled OLS): Baseline with class dummies and linear year trend.
    Residual diagnostics revealed systematic agent-level prediction errors
    (mean residuals ranging from -1.5 to +0.5 across agents), motivating
    the addition of agent fixed effects.

  Model 2 (FE + Interactions): Added agent fixed effects, class × channel
    interaction terms, and the dtca_zero regime indicator. Substantial
    improvement in fit by absorbing brand-specific baselines and allowing
    heterogeneous marketing response slopes. Uses continuous year trend
    to enable out-of-sample prediction.

  Model 3 (Mixed Effects): Replaced agent fixed effects with random
    intercepts. More parameter-efficient; handles cold-start agents (Pexeva)
    by assigning the population-level intercept. Comparable in-sample fit
    with fewer effective parameters.

FEATURE ENGINEERING:
  - Log transformations (log1p) for sales, detailing, and DTCA
  - dtca_zero binary regime indicator (47% of observations)
  - Continuous year_trend replacing categorical year dummies
  - Class × channel interaction terms via formula API
  - Agent whitespace stripping to prevent Patsy encoding errors

KEY FINDINGS:
  - Detailing elasticities are consistently higher than DTCA across all classes
  - PPI shows the highest detailing responsiveness
  - Statins show the strongest DTCA response among the three classes
  - Marginal ROI varies substantially by agent spend level, confirming
    diminishing returns above the $80-93M detailing saturation threshold
  - The dtca_zero regime indicator captures the structural difference between
    dual-channel and physician-only commercial models

NEXT STEP:
  The best parametric model advances to Notebook 04 for the champion comparison
  against the best ML model from Notebook 03.
```

In [29]:

```python
# Save predictions and metrics for downstream comparison
roi_df.to_csv("data/parametric_predictions.csv", index=False)
comparison.to_csv("data/parametric_benchmark.csv", index=False)
elasticity_df.to_csv("data/parametric_elasticities.csv", index=False)

print("Saved: data/parametric_predictions.csv")
print("Saved: data/parametric_benchmark.csv")
print("Saved: data/parametric_elasticities.csv")
```

```
Saved: data/parametric_predictions.csv
Saved: data/parametric_benchmark.csv
Saved: data/parametric_elasticities.csv
```

In [ ]: